CSE 202

Algorithm Design and Analysis

Russell Impagliazzo
russell@cs.ucsd.edu

Lecture 10:, greedy algorithms ``achieve the bounds''proofs', greedy approximation algorithms

THE GREEDY METHOD

- In some cases (not all!!!!!), there is sufficient structure that allows you to reach the correct solution by just picking the straightforward “best” decision at each stage.

- This is called the Greedy Method.
- It doesn’t always work.

- Just as in life, acting in one’s immediate best interest is not always the best longer-term strategy.

THE GREEDY METHOD REQUIRES PROOF

- The Greedy Method does not always work, even when it is ``obvious'' that it does. Because of this, in order to use the Greedy Method, we must prove the correctness of the algorithm. Intuitive arguments are very likely incorrect.

IMMEDIATE BENEFIT VS OPPORTUNITY COSTS

IMMEDIATE BENEFIT: How does the choice we're making now contribute to the objective function

OPPORTUNITY COST: How does the choice we're making now restrict future choices?

Greedy (usually) Take best immediate benefit and ignore opportunity costs.

Greedy is optimal: Best immediate benefits outweigh opportunity costs

TECHNIQUES TO PROVE OPTIMALITY

We'll see a number of general methods to prove optimality:

1. Modify the solution , aka Exchange, Transformation: most general
2. Greedy-stays-ahead: more intuitive
3. Greedy achieves the bound: also comes up in approximation, LP, network flow-- Today
4. Unique local optimum: dangerously close to a common fallacy

Which one to use is up to you, but only Modify-the-solution applies universally, others can be easier but only work in special cases

ACHIEVES-THE-BOUND

This is a proof technique that does not work in all cases.

The way it works is to argue that, when the greedy solution reaches its peak cost, it reveals a ***bound***

Then show this bound is also a lower bound on the cost of any other solution

So we are showing : Cost(GS) ≤ Bound ≤ Cost (OS)

Allows the two inequalities to be separated

GREEDY APPROXIMATION ALGORITHMS

Sometimes, we still want to use greedy algorithms (or other algorithms) when they do not give optimal solutions

Maybe finding optimal solutions is NP-complete, or the greedy algorithm is much, much faster than an exactly optimal algorithm for the problem

We can sometimes give approximation guarantees:

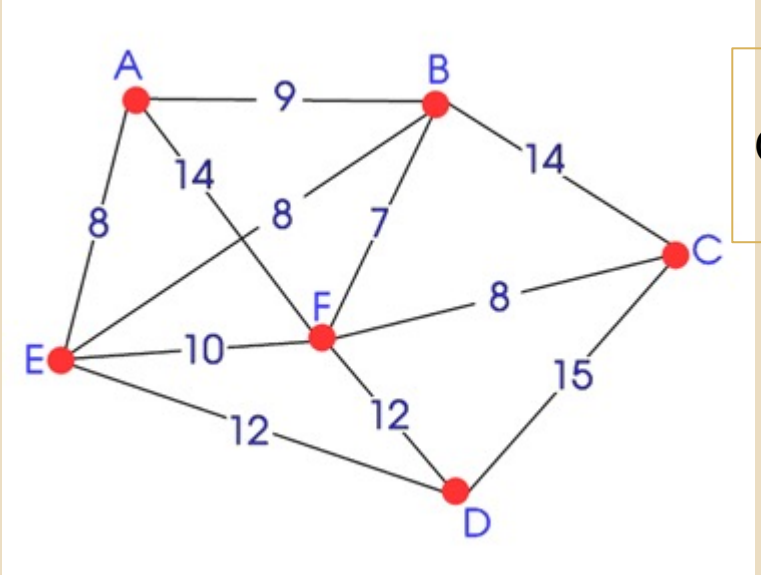
Cost(GS) ≤ C Cost(OS)

C* Value(GS) ≥ Value(OS)

C is called approximation ratio

TRAVLELING SALESPERSON PROBLEM

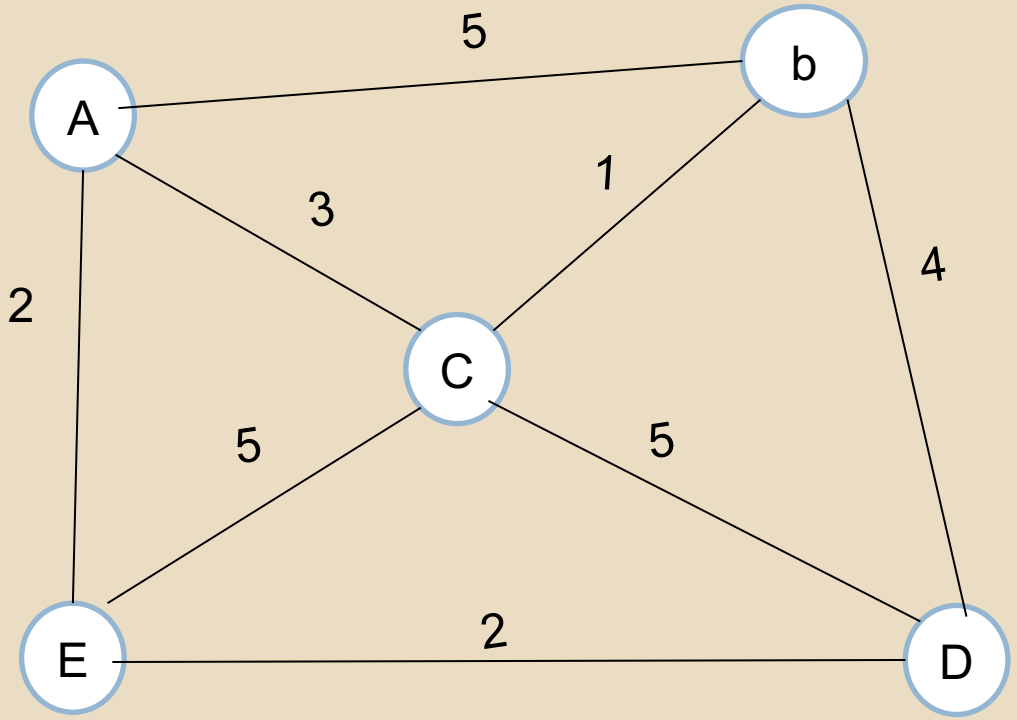
Salesperson needs to visit
Every location and return
To start,
Wants to minimize total distance
Travelled
Famous NP-complete problem



diCan assume stances
Obey triangle inequality:
$D(u,v) \leq D(u,w)+D(w,v)$

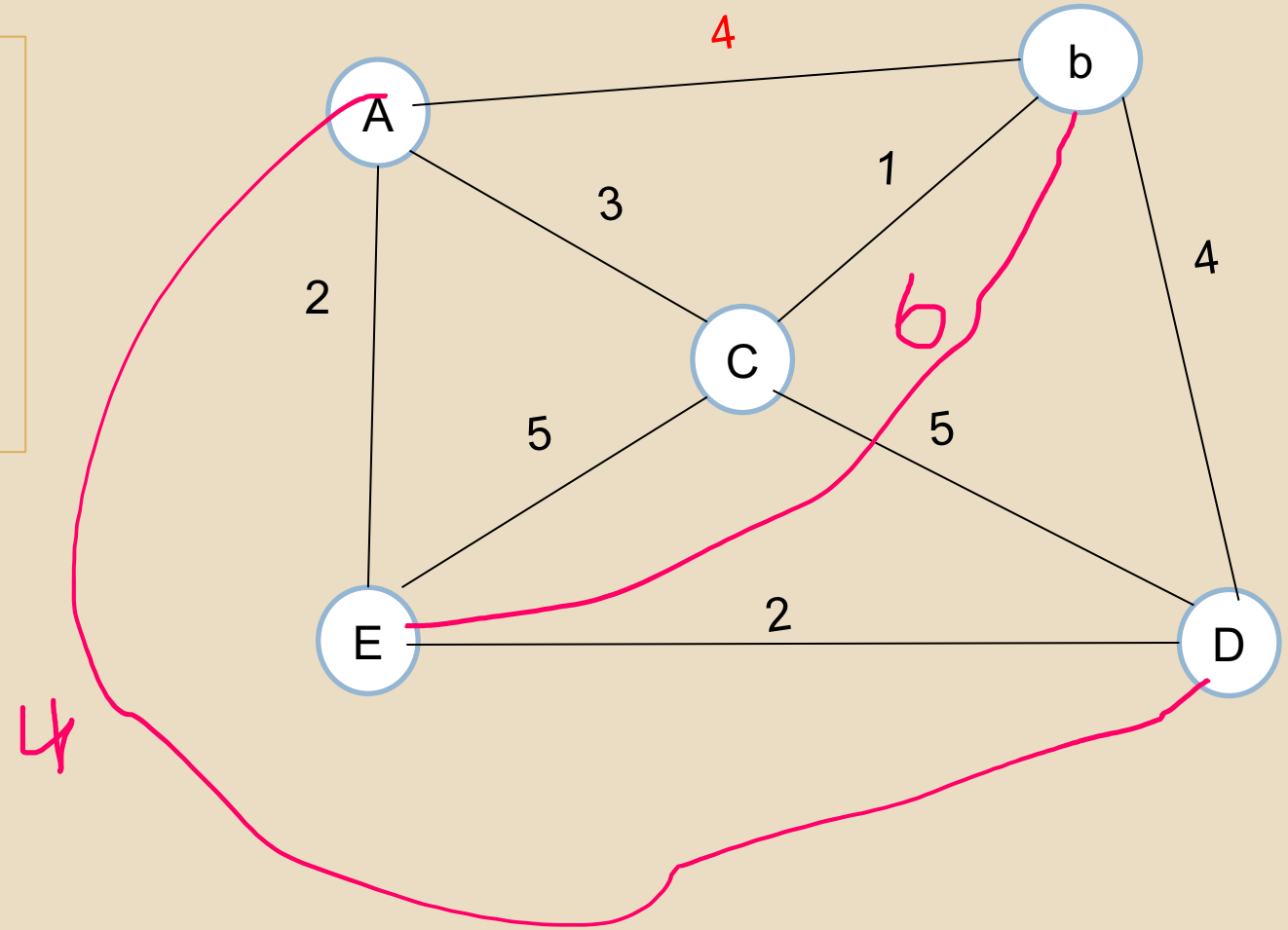
TRAVLELING SALESPERSON PROBLEM

Salesperson needs to visit
Every location and return
To start,
Wants to minimize total distance
Travelled
Famous NP-complete problem



TSP APPROXIMATION

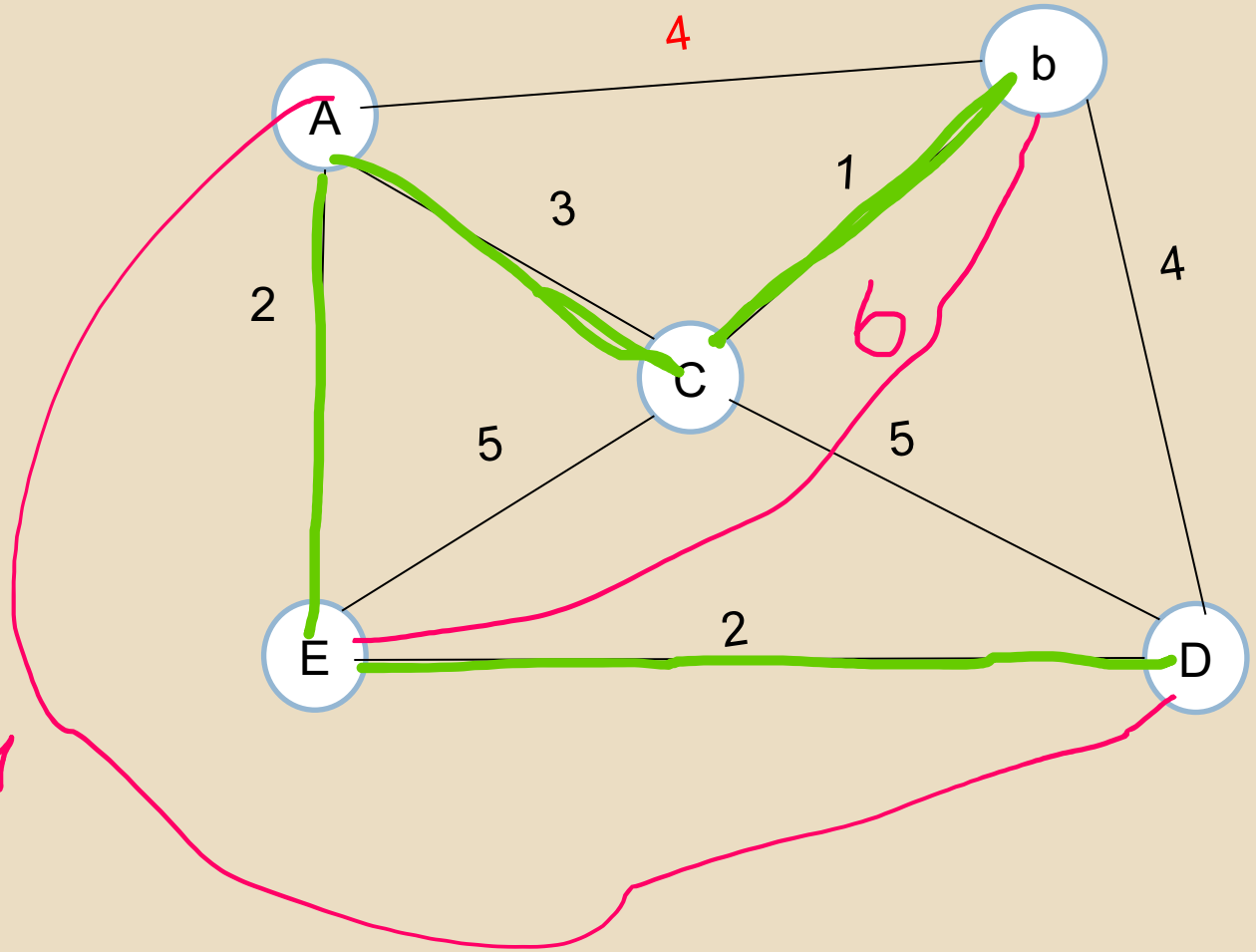
Step 1 : compute all pairs shortest path distances .
Distances now obey triangle inequality: D(x,y) $\leq D(x,z) + D(z,y)$



TSP APPROXIMATION

Step 1 : compute all pairs shortest path distances .
Distances now obey triangle inequality: D(x,y) $\leq D(x,z) + D(z,y)$

Step 2:Find Min. spanning tree

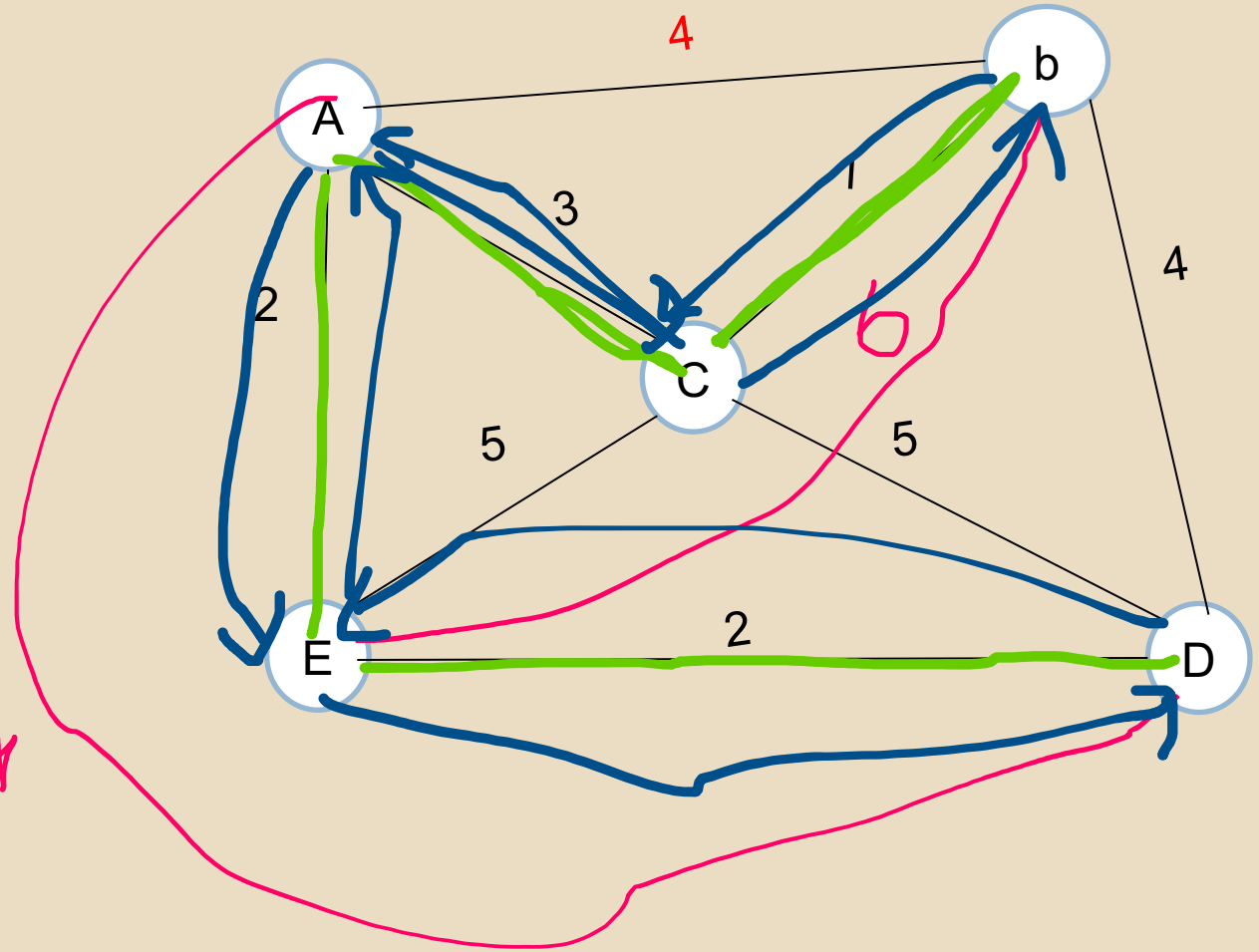


TSP APPROXIMATION

Step 1 : compute all pairs shortest path distances .
Distances now obey triangle inequality: D(x,y) $\leq D(x,z) + D(z,y)$

Step 2:Find Min. spanning tree

Step 3: Find in-order traversal of spanning tree: A→ E→D→E→ A→C→B→C→ A



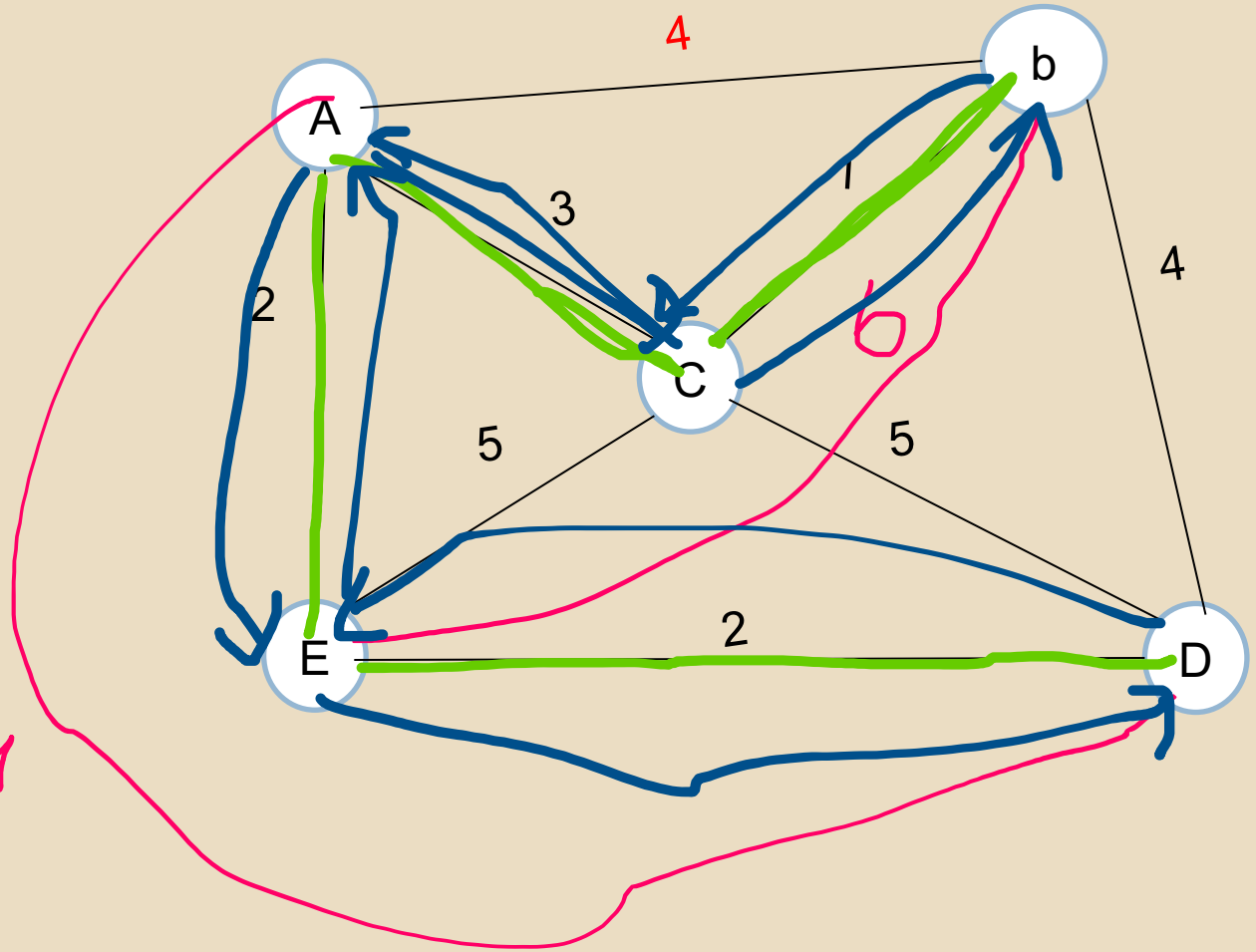
TSP APPROXIMATION

Step 1 : compute all pairs shortest path distances .
Distances now obey triangle inequality: D(x,y) ≤ $D(x,z) + D(z,y)$

Step 2:Find Min. spanning tree

Step 3: Find in-order traversal of spanning tree: A→ E→D→E→ A→C→B→C→ A

Step 4: short-circuit: A→E→D→C→B→A



RELATIONSHIP BETWEEN MST AND TSP

Let TSP be the cost of the minimum travelling salesperson tour

MST be the cost of the minimum spanning tree

Claim: $TSP > MST$. Proof: the edges used in the tour connect all the vertices. (Strict inequality because it's also a cycle)

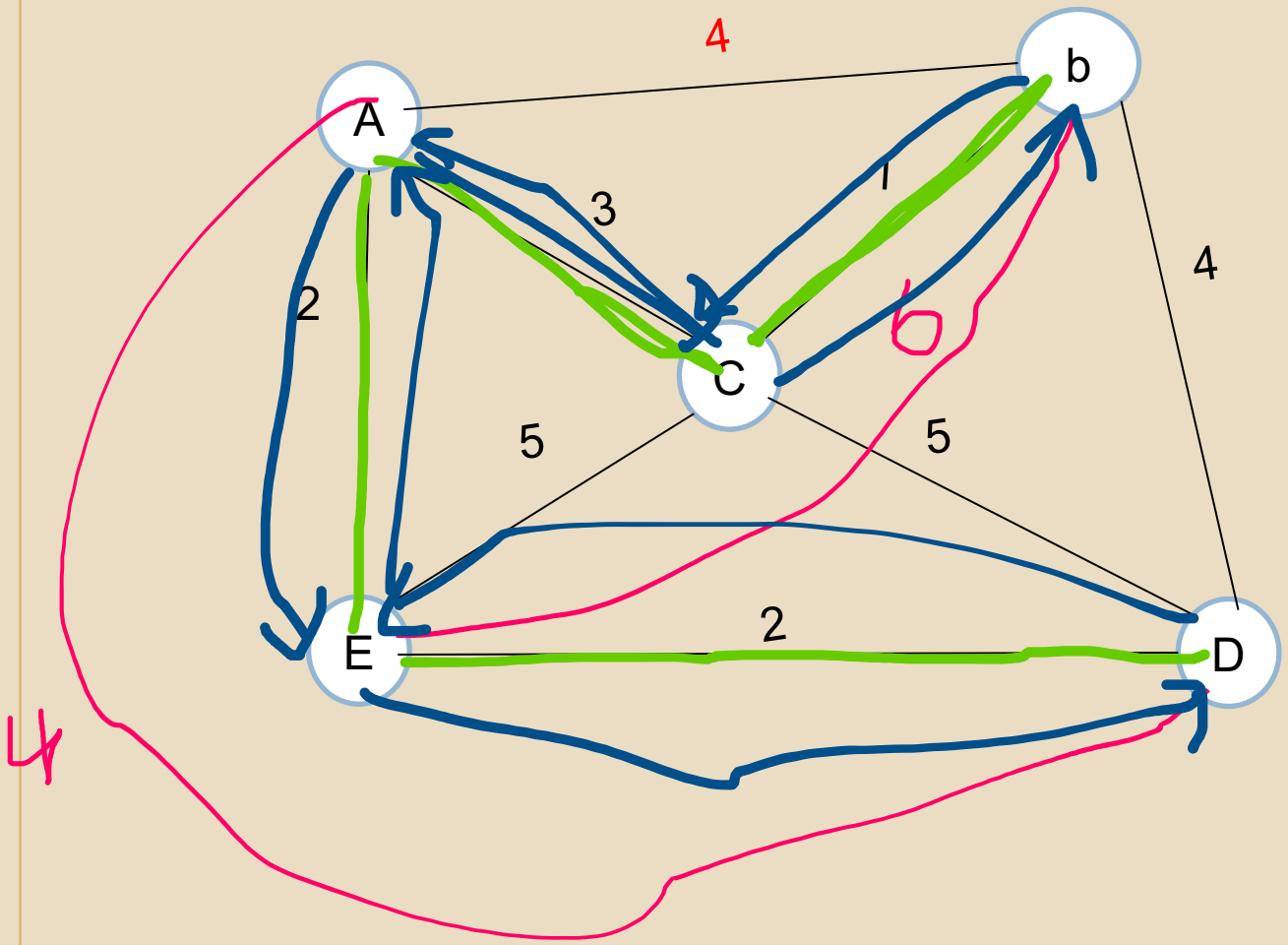
TSP APPROXIMATION

Step 1 : compute all pairs shortest path distances .
Distances now obey triangle inequality: D(x,y) ≤ $D(x,z) + D(z,y)$. Doesn't change MST, TSP.

Step 2:Find Min. spanning tree.
Cost: MST

Step 3: Find in-order traversal of spanning tree: A→ E→D→E→ A→C→B→C→ A.
Cost 2MST

Step 4: short-circuit:
A→E→D→C→B→A.
Cost : < 2 MST from TI

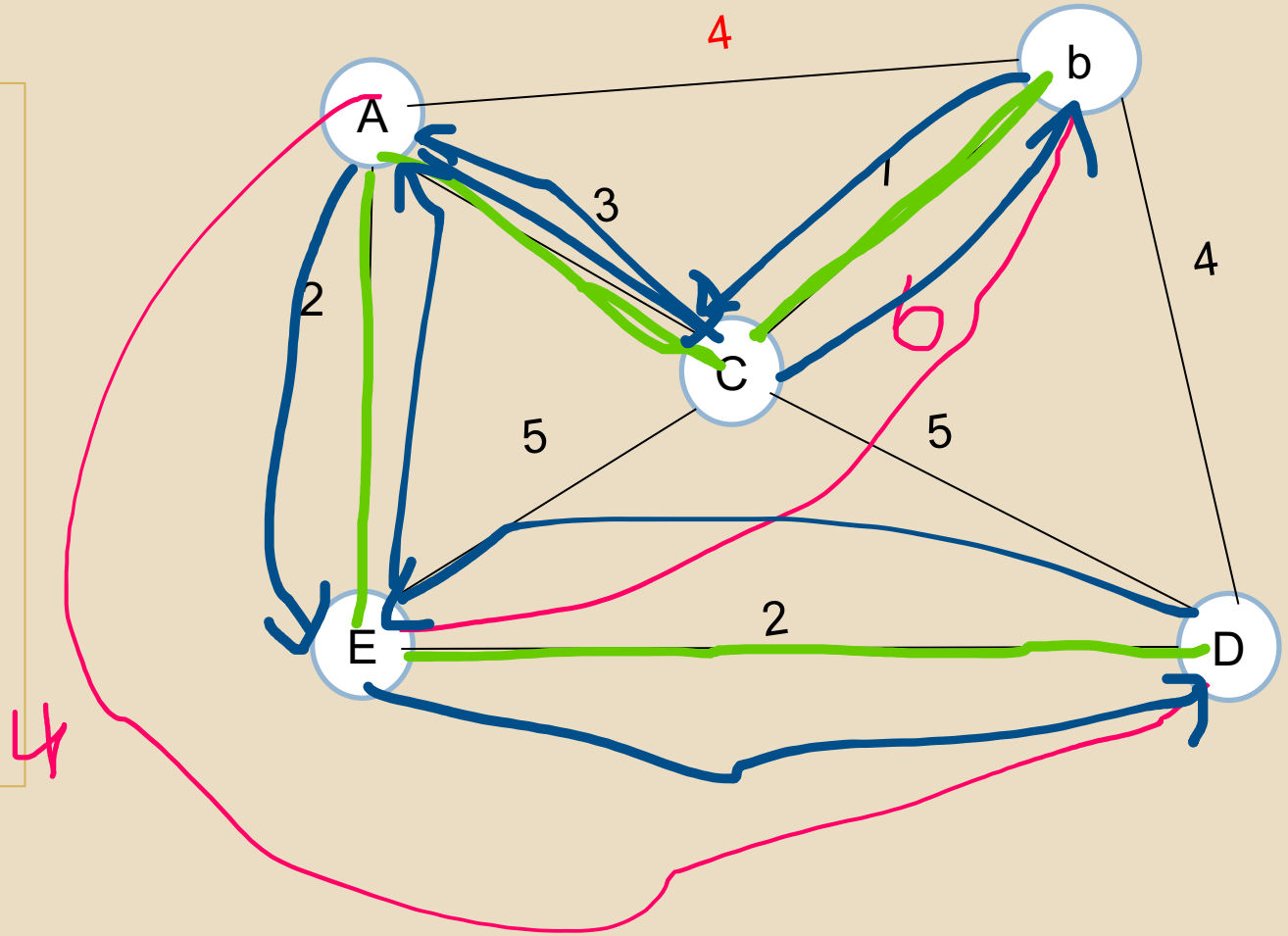


TSP APPROXIMATION

Step 4: short-circuit:
A→E→D→C→B→A.
Cost : < 2 MST from TI

Since MST < TSP,
OurCost< 2 TSP.

We are within factor of 2 of the optimal.



GENERAL METHOD: COMBINATION OF BOUNDS

Achieve-the-bound proof: (minimization)

Define Bound or obstacle for a solution, LB

Show really a bound, OPT$\geq LB$

Show greedy algorithm achieves bound, Cost(GS) = LB

COMBINATION OF BOUNDS: simple version:

Replace last step with Cost(GS) $\leq C * LB$, C is approximation ratio

COMBINATION OF BOUNDS: more complex version:

Do steps 1,2, for $LB_1, LB_2, LB_3, \ldots LB_k$

Show Cost(GS) $\leq C_1 LB_1 + C_2 LB_2 + \cdots C_k LB_k \leq (C_1 + \cdots C_k) OPT$

COMBINATION OF BOUNDS : TSP

Bound : Minimum spanning tree cost, MST

Bounding lemma: $TSP \geq MST$

Achieves a multiple of bound: $Cost(GS) \leq 2 * MST$

Conclusion: GS is a factor of 2 approximation to optimal TSP

LOAD BALANCING

Many jobs requiring time $t_1, .. t_n$ need to be divided between two Identical machines.

We want to complete the jobs as early as possible, so want to minimize the greater of the two total times of jobs assigned to the two machines.

For example, if the times are 11, 7, 6, 5, 4, 3

We could give one machine 11, 3 and 4

The other 6,5, and 7

Both would finish in 18 time steps

M1: 11, 5, 3= 19

M2: 7,6 , 4, =17

LOAD BALANCING

Many jobs requiring time $t_1, .. t_n$ need to be divided between two Identical machines.

We want to complete the jobs as early as possible, so want to minimize the greater of the two total times of jobs assigned to the two machines.

For example, if the times are 11, 7, 6, 5, 3, 4,

We could give one machine 11, 3 and 4

The other 6,5, and 7

Both would finish in 18 time steps

GREEDY ALGORITHM

Sort the jobs from longest to shortest.

In order, assign each job to the machine with less load

3,3,2,2,2; M1: 3,3, M2: 2, 2,2

M1: 3.2=5

M2: 3.2, 2=7

OPT 3,3=6/ 2,2,2=6

Ratio: 7/6

So greedy is not optimal.

USE LOWER BOUNDS

Use lower bounds on OPT, the optimal finish time

What is a lower bound on OPT?

LB1= ½ $\sum t_i$

LB2= $Max_i\ t_i = t_1$

USE LOWER BOUNDS

Use lower bounds on OPT, the optimal finish time

What is a lower bound on OPT?

Total = $\sum t_i$

Max = $\max t_i$

OPT ≥ ½ Total

OPT ≥ Max

2OPT BOUND

Theorem: Cost(GS) ≤ 2OPT:

In fact, this is true for any solution, since Cost(any solution) ≤ Total ≤ 2*OPT

So let’s try to do better

BETTER BOUND

Let T_1 be the larger loaded side of the greedy algorithm, and T_2 the smaller side. Then $T_1+T_2 = Total$ and $T_2 \leq T_1$, so $T_2 \leq \frac{1}{2}Total \leq OPT$

Let t_j, the j'th largest job, be the last job added to T_1 's side. Before it was added, that side had the smaller load, so $T_1 - t_j \leq \frac{1}{2}\left(Total - t_j\right)$

$Cost(GS) = T_1 \leq \frac{1}{2}\left(Total + t_j\right) \leq OPT + \frac{1}{2}t_j \leq OPT + \frac{1}{2}Max \leq \frac{3}{2}\ OPT$

Can do somewhat better. If j=1, $T_1 = Max = \mathrm{OPT}$. Can we have j=2?

EVEN BETTER BOUND

Let t_j, the j'th largest job, be the last job added to T_1 's side. Before it was added, that side had the smaller load, so $T_1 - t_j \leq \frac{1}{2}\left(Total - t_j\right)$

If $j = 1, Cost(GS) = t_1 = Max = OPT$

j =2 is impossible

$$Cost(GS) = T_1 \leq \frac{1}{2}\left(Total + t_j\right) \leq OPT + \frac{1}{2}t_j$$

Because there are j jobs at least as large as t_j , $Total \geq j\ t_j$, so $\frac{1}{2}t_j \leq \frac{1}{2j}Total \leq \frac{1}{j}OPT$

$$Cost(GS) \leq \left(1 + \frac{1}{j}\right)OPT, j \geq 3, so\ Cost(GS) \leq \frac{4}{3}OPT$$

EVEN BETTER THAN THAT

What happens if j=3?

t_1 on one side, t_2+t_3 on the other, then more jobs go on the first, but t_2+t_3 stays the largest.

Of the three largest jobs, t_1,t_2,t_3, two must go on the same side in any schedule. Therefore, OPT is at least t_2+t_3=GS in this case

If j is at least 4, the previous argument gives is at most $GS \leq OPT + \frac{1}{j}OPT = \frac{5}{4}OPT$

CAN WE KEEP GOING?

approximation ratio is at least 7/6.

7/6 \le Approx Ratio \le 5/4

With a lot of case analysis, ApproxRatio=7/6

3,3, 2,2,2

OPT 3+3, 2+2+6

GS 3+2+2=7 , 3+2=5

CAN WE KEEP GOING?

approximation ratio is at least 7/6 for greedy algorithm as described. So let's change the algorithm.

Analysis shows: Bad case is when bottleneck is for small number of large jobs, e.g., few large jobs determine cost.

Maybe we need to treat the first k jobs in a special way, for a small value of k. What could we do for the first k jobs?

EXHAUSTIVE SEARCH

Takes exponential time. Can't do it for general, but could do it for first k, in $O(2^k)$ time.

Let OPT_k be the optimal solution for the k largest jobs. Clearly, $OPT \geq Cost(OPT_k)$

Then use the greedy algorithm starting from OPT_k for the remaining jobs.

If the less loaded machine in OPT_k stays the less loaded machine to the end, then $Cost(GS) = Cost(OPT_k) = OPT$

IF NOT

If not, then some job $j > k$ is the last one added to the higher loaded machine in GS.

Then $Cost(GS) \leq OPT + \frac{1}{2}t_j \leq OPT + \frac{1}{2j}Total \leq \left(1 + \frac{1}{j}\right)OPT < \left(1 + \frac{1}{k}\right)OPT$

What value can we afford to pick for k?

FINAL ALGORITHM

Pick k= log n. Time for exhaustive search is O(n).

Approximation ratio = 1+ 1/log n , gets closer to OPT as n increases

We'll see an even better ``approximation scheme'' when we look at Dynamic programming

PARALLEL JOB SCHEDULING

We have P processors.

Each of n jobs J_i requires p_i processors for time t_i and must be run with all processors until finished.

We want to minimize the makespan, the time when all jobs are finished.

Previous problem is special case when P=2, p_i=1

GREEDY ALGORITHM

Here's a (seemingly wasteful) greedy algorithm

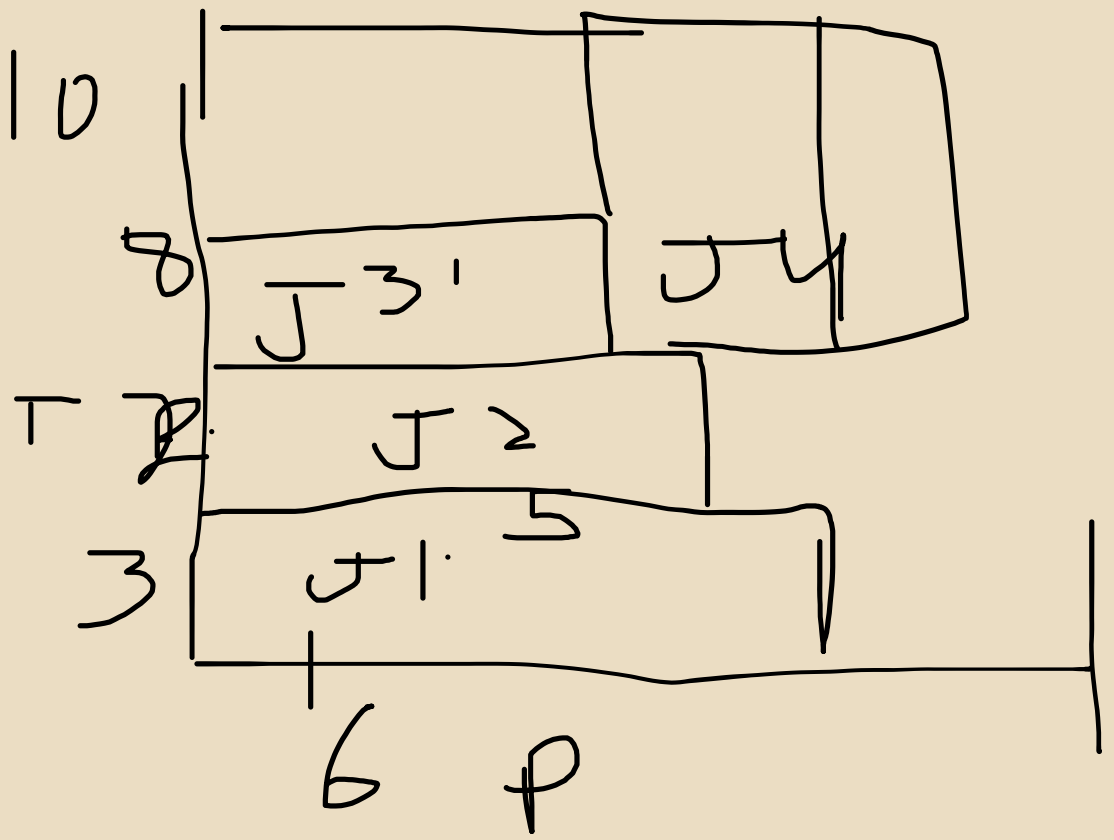
Sort jobs from largest processor requirement to smallest.

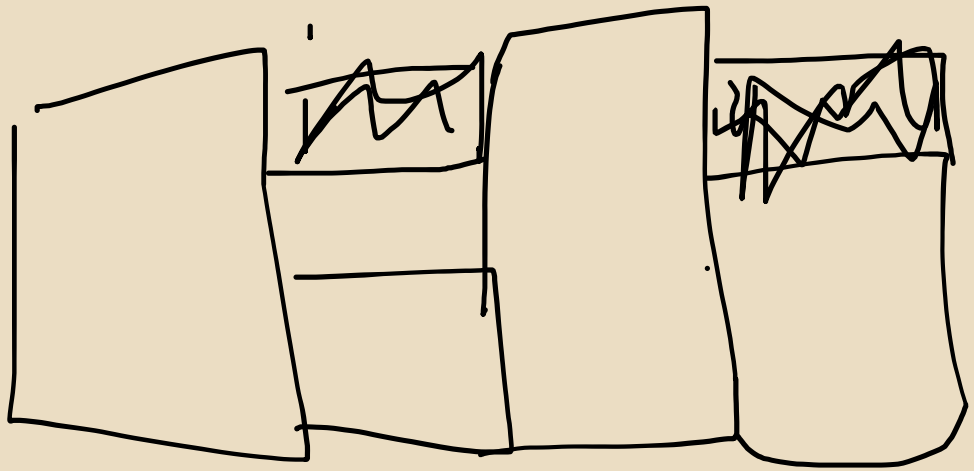
At time 0, schedule jobs in that order until the next does not fit.
Repeat for unscheduled jobs whenever a job finishes.
P=8 , (6,3), (5,2), (4,3), (2, 5)

T=0 schedule (6,3)

T=3 schedule (5,2)

T=5 schedule (4,3), (2,5) T=10


10
8
J3
J4
J2
5
3
J1
6
p



PARALLEL JOB SCHEDULING

We have P processors.

Each of n jobs J_i requires p_i processors for time t_i and must be run with all processors until finished.

We want to minimize the makespan, the time when all jobs are finished.

Previous problem is special case when P=2, p_i=1

GREEDY ALGORITHM

Here's a (seemingly wasteful) greedy algorithm

Sort jobs from largest processor requirement to smallest.

At time 0, schedule jobs in that order until the next does not fit.
Repeat for unscheduled jobs whenever a job finishes.

LOWER BOUNDS

What are the analogs of our previous lower bounds?

LB like Total/2 Define $Work = \sum t_i p_i$. Then $\frac{Work}{P}$ is a lower bound on the total time we need.

LB like Max. Maxtime = $\max_i t_i$ is a lower bound

TWO STAGES

Divide the time our schedule takes into two stages:

T_1 = Time until the last job starts

T_2 = Time after the last job starts

GT = $T_1 + T_2$

FIRST STAGE KEEPS PROCESSORS BUSY

Lemma: While there is an unscheduled job, the number of idle processors < P/2 .

Let p_j be the number of processors required by the next job to be scheduled. The number of idle processors < p_j, or we would schedule the next job. So if $p_j \leq \frac{P}{2}$ the claim follows. If $p_j > \frac{P}{2}$, then so is the requirement for every previous job, so there must be a job using more than half the processors currently running, leaving fewer than half the processors idle

BOUNDING THE RATIO

By the lemma, $Work \geq \frac{P}{2}(T_1), or\ T_1 \leq 2\frac{Work}{P} \leq 2\ OPT$

Since the last job to finish has already started at the beginning of the second phase, T_2 $\leq Maxtime \leq OPT$

Therefore, GT =$T_1 + T_2 \leq 2\ OPT + OPT = 3OPT$.

CAN WE DO BETTER?

Divide T_1 into time while jobs are > P/2 requirements and after last such job finishes, $T_{\{11\}}$, $T_{\{12\}}$

Claim: $OPT \geq T_\{11\}$ because we can do at most one such job at a time.

Claim: After last larger than half job finishes and before all have started, at most P/3 idle processors

Proof: at least two jobs at all times. If p_j < P/3, as before, done.

If p_j > P/3, so are the two jobs running, so > 2P/3 processors used.

CONCLUSION

GT=T_{11}+T_{12}+T_2.

Work > P/2 T_{11} +2/3 P T_{12}, so LB_1 > ½ T_{11} +2/3 T_{12}
3/2 LB_1 > ¾ T_{11}+T_{12}

$$GT \leq \frac{3}{2} LB_1 + \frac{1}{4} T_{\{11\}} + T_2 \leq \left(\frac{3}{2}\right) OPT + \left(\frac{1}{4}\right) OPT + OPT = 2.75\ OPT$$

APPROXIMATION SUMMARY

Sometimes, we can come very close to optimal, sometimes we just aren't horribly worse. Whether approximation is appropriate depends on application.

Best approximation algorithms not always ``Best candidate for exact solution''. Instead, it gives the best balance between different extreme cases , hedging our bets.